

Prof. Dr. Peer Küppers

DSCB140
Datenbanken und Datenkunde I

VL6 – Structured Query Language (SQL)
Teil 3: DQL

Agenda

- **Query-Struktur und einfache Abfragen**
- ☐ **WHERE-Ausdrücke**
- ☐ **Berechnete Spalten und Sortierung**
- ☐ **Gruppierung und Aggregation**
- ☐ **Kombination von Tabellen: JOIN und UNION**
- ☐ **Subqueries und Common Table Expressions**
- ☐ **Zusammenfassung und Ausblick**

Selektieren von Daten

DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Daten werden mittels des SELECT-Statements abgerufen.

```
SELECT [DISTINCT] spaltenliste  
FROM tabellenliste  
[WHERE bedingungsausdruck]  
[GROUP BY spaltenliste]  
[HAVING bedingungsausdruck]  
[ORDER BY spaltenliste]  
[LIMIT n]
```

- Dieses erlaubt uns
 - einfache Abfragen (bspw. aller Zeilen einer Tabelle),
 - dynamische Berechnungen (bspw. preis*menge zur Umsatzberechnung),
 - die Suche nach einzelnen Datensätzen (eingeschränkt mittels WHERE-Bedingung),
 - die Gruppierung und Aggregation von Daten (bspw. Umsatz pro Produktkategorie)
 - und vorab auch die Kombination normalisierten Tabellen durch Joins (bspw. Anzeigen des Kategorienamens für selektierte Produkte).

Selektieren von Daten

```
SELECT [DISTINCT] spaltenliste  
FROM tabellenliste  
[WHERE bedingungsausdruck]  
[GROUP BY spaltenliste]  
[HAVING bedingungsausdruck]  
[ORDER BY spaltenliste]  
[LIMIT n]
```

- Spaltenangabe: entweder mit * alle Spalten oder durch Komma getrennt einzelne Spalten zurückgeben.

- Optionale Angabe:
DISTINCT

- Duplikate in der Ausgabe entfernen, d.h. nur eindeutige Werte anzeigen lassen.

```
SELECT Name  
FROM Person
```

Name
Smith
Jones
Brown
Jones
Brown

```
SELECT DISTINCT Name  
FROM Person
```

Name
Smith
Jones
Brown

Selektieren von Daten

```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

- Spaltennamen können umbenannt werden:
SELECT name **as n** FROM Person

n
Smith
Jones
Brown
Jones
Brown

- Spalten können dynamisch berechnet werden:
SELECT preis, menge,
 preis*menge as umsatz
FROM Sales

preis	menge	umsatz
1.1	10	11.0
2.0	30	60.0
0.8	50	40.0

Selektieren von Daten

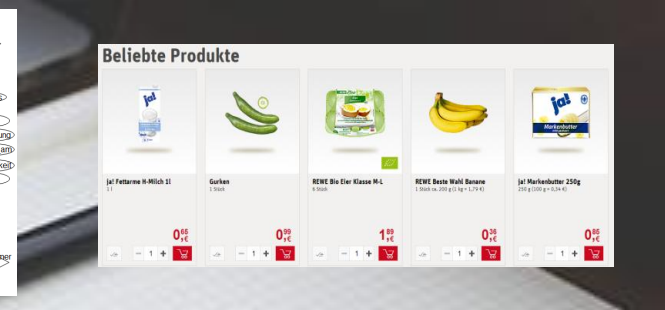
```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

- Die Anzahl Zeilen kann mittels LIMIT n eingeschränkt werden.
- Die Sortierung kann mittels ORDER BY definiert werden.
 - Es kann nach mehreren Spalten sortiert werden.
 - Im Standard wird aufsteigend nach der/den nach „ORDER BY“ angegebenen Spalte(n) sortiert.
 - Soll absteigend sortiert werden muss hinter der jeweiligen Spalte „DESC“ angegeben werden.

```

    erDiagram
        Benutzer ||--o{ Kunde : "is-a"
        Benutzer ||--o{ Mitarbeiter : "is-a"
        Benutzer ||--o{ Bestellung : "hat"
        Bestellung ||--o{ Produkt : "hat"
        Produkt ||--o{ Kategorie : "gehört zu"

        Benutzer {
            string idnr
            string straße
            string taururname
            string plz
            string ort
            date geburtsdatum
        }
        Kunde {
            string rechnung
            string abteilung
        }
        Mitarbeiter {
            string mnr
            string telefonnr
        }
        Bestellung {
            string bestnr
            date bestelldatum
        }
        Produkt {
            string pnr
            string bezeichnung
            float eingangspreis
            float gebietspreis
            float preis
        }
        Kategorie {
            string kalkul
            string bezeichnung
        }
  
```



- DQL 1.1 ■ Selektieren Sie alle Spalten und alle Zeilen aus der Tabelle „produkt“.
- DQL 1.2 ■ Selektieren Sie die Spalten „bezeichnung“ und „preis“ aus der Tabelle „produkt“.
- DQL 1.3 ■ Selektieren Sie die Spalten „bezeichnung“ und „preis“ aus der Tabelle „produkt“. Sortieren Sie absteigend nach Preis und limitieren Sie die Ausgabe auf drei Zeilen, um die „Top 3 der teuersten Produkte“ anzuzeigen.
- DQL 1.4 ■ Zeigen Sie die Orte an, aus denen die Kunden kommen (ohne Duplikate).
- DQL 1.5 ■ Zeigen Sie pro Produkt den Lagerwert ($\text{preis} * \text{verfuegbarkeit}$) an.

Agenda

- ☐ Query-Struktur und einfache Abfragen
- ☒ **WHERE-Ausdrücke**
- ☐ Gruppierung und Aggregation
- ☐ Kombination von Tabellen: JOIN und UNION
- ☐ Subqueries und Common Table Expressions
- ☐ Zusammenfassung und Ausblick

Einschränken von Ergebnismengen

```
SELECT [DISTINCT] spaltenliste  
FROM tabellenliste  
[WHERE bedingungsausdruck]  
[GROUP BY spaltenliste]  
[HAVING bedingungsausdruck]  
[ORDER BY spaltenliste]  
[LIMIT n]
```

- Oft ist man nicht an allen Zeilen, sondern nur bestimmten Kriterien entsprechenden Daten in Tabellen interessiert.
- Mit dem WHERE-statement können die zurückgegebenen Zeilen eingeschränkt werden.
 - In der DML wird das WHERE-statement identisch für UPDATE/DELETE verwendet.

Einschränken von Ergebnismengen

- **„WHERE-Basisprädikate“ (einfache Suchbedingungen)**
 - Vergleichsoperatoren (=, !=, >, >=, <, <=)
 - Vergleich von zwei Ausdrücken, z.B.:
`select * from produkt where preis > 3` oder `... where bezeichnung = 'Banane'`
 - *Im Beispiel: Ausdrücke sind ein Identifier (bspw. Spalte preis) und ein literal (bspw. 3)*
 - Bereichsvergleich (`BETWEEN`)
 - Prüfung, ob ein Wert in einer „range“ liegt, z.B.: `... where preis between 0.8 and 1.3`
 - Mengenzugehörigkeit (`IN`)
 - Prüfung, ob ein Wert in einer Menge liegt, z.B.:
`... where preis in (11, 15, 1)` oder `... where bezeichnung in ('Zitrone', 'Banane')`
 - NULL-Prüfung (`IS NULL`)
 - Prüfung, ob ein Wert gesetzt wurde oder nicht, z.B.: `... where katid is null`
 - Mustersuche / Pattern Matching (`LIKE`)
 - Prüfung, ob ein Wert einem Muster (bspw. mit Platzhaltern) entspricht, z.B.:
`... where bezeichnung like '%schokolade'`
 - Mustervergleiche können sehr komplex gestaltet werden (insb. über reguläre Ausdrücke)

Einschränken von Ergebnismengen

■ Vergleichsoperatoren

```
select *  
from produkt  
where preis > 1.2
```

Operator	Beschreibung	Beispiel
=	Gleich	Nachname = 'Müller'
<> oder !=	ungleich	Lagerbestand <> 0
>	Größer	Preis > 20.0
<	Kleiner	Preis < 20.0
>=	größer gleich	Preis >= 20.0
<=	kleiner gleich	Preis <= 20.0

- **Alle** Ausdrücke sind mit **logischen Operatoren** verknüpfbar: **OR, AND**
 - AND bindet stärker als OR (ggf. sind somit Klammern um Ausdrücke nötig)
- **Alle** Ausdrücke können invertiert werden: **NOT**
 - NOT bindet stärker als AND und OR (ggf. sind somit Klammern um Ausdrücke nötig)
- Hier ein Beispiel mit Vergleichsoperatoren:

```
select * from produkt  
where preis > 1.2 and  
not (verfuegbarkeit < 15 or verfuegbarkeit > 100)
```

Einschränken von Ergebnismengen

■ Bereichsvergleich

- Die Randelemente sind im Vergleich enthalten. Folgende Statements liefern somit dieselbe Ergebnismenge:

```
select * from produkt
where preis between 0.8 and 1.3
```

```
select * from produkt
where preis >= 0.8 and preis <= 1.3
```

■ Mengenzugehörigkeit

- Mit „IN“ können Mengenzugehörigkeit geschickt geprüft werden. Folgende Statements liefern dieselbe Ergebnismenge:

```
select *
from produkt
where bezeichnung in ('Zitrone', 'Banane')
```

```
select *
from produkt
where bezeichnung = 'Zitrone' or bezeichnung = 'Banane'
```

- Insbesondere lässt sich die Vergleichsmenge dynamisch erzeugen (Subqueries)
 - Dazu später mehr.

Einschränken von Ergebnismengen

■ NULL-Prüfung

- Mittels Invertierung „**NOT NULL**“ erhält man nur Zeilen, bei denen der entsprechende Wert gesetzt ist.

```
select *
from produkt
where not katid is null
```

■ Pattern Matching

- **LIKE**-Operator unterscheidet nicht zwischen Groß- und Kleinschreibung („=“ schon*)
- **LIKE** erlaubt Platzhalter: % = kein oder beliebig viele Zeichen, _ = genau ein Zeichen
 - Beispiel: Produkte mit „schoko“ in der Bezeichnung bzw. genau einem Zeichen vor „anane“

```
select * from produkt
where bezeichnung like '%schoko%'
```

```
select * from produkt where
bezeichnung like '_anane'
```

- Reguläre Ausdrücke sind noch mächtiger und können mittels **REGEXP**
 - <https://dev.mysql.com/doc/refman/5.7/en/pattern-matching.html>
 - Beispiel: Produkte mit Leerzeichen in der Bezeichnung, die mit einem „e“ oder „E“ beginnen.
 - Dies behandeln wir nicht im Detail.

```
select * from produkt where
bezeichnung regexp
'^[eE].*[:space:].*';
```

Einschränken von Ergebnismengen

■ Bisher behandelt:

■ „WHERE-Basisprädikate“ und -ausdrücke (einfache Suchbedingungen)

- Vergleichsoperatoren (*comparison_operator*)
 - Darin: Vergleiche zwischen Werten (*literal*) und Spalten (*identifier*) anhand einfacher Ausdrücke (*simple_expr*)
- BETWEEN, IN, LIKE und REGEXP

■ Kombination von Ausdrücken durch logische Operatoren

- OR (bzw. ||)
- AND (bzw. &&)
- NOT (bzw. !)

Logische Operatoren

```
expr:
  expr OR expr
| expr || expr
| expr XOR expr
| expr AND expr
| expr && expr
| NOT expr
| ! expr
| boolean_primary IS [NOT] {TRUE | FALSE | UNKNOWN}
| boolean_primary

boolean_primary:
  boolean_primary IS [NOT] NULL
| boolean_primary <=> predicate
| boolean_primary comparison_operator predicate
| boolean_primary comparison_operator {ALL | ANY} (subquery)
| predicate

comparison_operator:
  = | >= | > | <= | < | <> | !=

predicate:
  bit_expr [NOT] IN (subquery)
| bit_expr [NOT] IN (expr [, expr] ...)
| bit_expr [NOT] BETWEEN bit_expr AND predicate
| bit_expr SOUNDS LIKE bit_expr
| bit_expr [NOT] LIKE simple_expr [ESCAPE simple_expr]
| bit_expr [NOT] REGEXP bit_expr
| bit_expr

bit_expr:
  bit_expr | bit_expr
| bit_expr & bit_expr
| bit_expr << bit_expr
| bit_expr >> bit_expr
| bit_expr + bit_expr
| bit_expr - bit_expr
| bit_expr * bit_expr
| bit_expr / bit_expr
| bit_expr DIV bit_expr
| bit_expr MOD bit_expr
| bit_expr % bit_expr
| bit_expr ^ bit_expr
| bit_expr + interval_expr
| bit_expr - interval_expr
| simple_expr

simple_expr:
  literal
| identifier
| function_call
| simple_expr COLLATE collation_name
| param_marker
| variable
| simple_expr || simple_expr
| + simple_expr
| - simple_expr
| ~ simple_expr
| ! simple_expr
| BINARY simple_expr
| (expr [, expr] ...)
| ROW (expr, expr [, expr] ...)
| (subquery)
| EXISTS (subquery)
| {identifier expr}
| match_expr
| case_expr
| interval_expr
```

Einschränken von Ergebnismengen

■ Berechnungen sind in WHERE-Ausdrücken erlaubt

- Genau so wie bei der dynamischen Spaltenerzeugung (siehe oben).
- Beispiel: Prüfung auf berechneten Lagerwert

```
select * from produkt
where preis*verfuegbarkeit > 100
```

- Achtung: Spaltenalias sind nicht zulässig in WHERE-Bedingungen

```
select *,
preis*verfuegbarkeit as lagerwert
from produkt where lagerwert > 100
```

Spalte „lagerwert“ nicht bekannt für WHERE-Ausdruck

```
expr:
  expr OR expr
  | expr || expr
  | expr XOR expr
  | expr AND expr
  | expr && expr
  | NOT expr
  | ! expr
  | boolean_primary IS [NOT] {TRUE | FALSE | UNKNOWN}
  | boolean_primary

boolean_primary:
  boolean_primary IS [NOT] NULL
  | boolean_primary <=> predicate
  | boolean_primary comparison_operator predicate
  | boolean_primary comparison_operator {ALL | ANY} (subquery)
  | predicate

comparison_operator:
  = | >= | > | <= | < | <> | !=

predicate:
  bit_expr [NOT] IN (subquery)
  | bit_expr [NOT] IN (expr [, expr] ...)
  | bit_expr [NOT] BETWEEN bit_expr AND predicate
  | bit_expr SOUNDS LIKE bit_expr
  | bit_expr [NOT] LIKE simple_expr [ESCAPE simple_expr]
  | bit_expr [NOT] REGEXP bit_expr
  | bit_expr

bit_expr:
  bit_expr | bit_expr
  | bit_expr & bit_expr
  | bit_expr << bit_expr
  | bit_expr >> bit_expr
  | bit_expr + bit_expr
  | bit_expr - bit_expr
  | bit_expr * bit_expr
  | bit_expr / bit_expr
  | bit_expr DIV bit_expr
  | bit_expr MOD bit_expr
  | bit_expr % bit_expr
  | bit_expr ^ bit_expr
  | bit_expr + interval_expr
  | bit_expr - interval_expr
  | simple_expr

simple_expr:
  literal
  | identifier
  | function_call
  | simple_expr COLLATE collation_name
  | param_marker
  | variable
  | simple_expr || simple_expr
  | + simple_expr
  | - simple_expr
  | ~ simple_expr
  | ! simple_expr
  | BINARY simple_expr
  | (expr [, expr] ...)
  | ROW (expr, expr [, expr] ...)
  | (subquery)
  | EXISTS (subquery)
  | {identifier expr}
  | match_expr
  | case_expr
  | interval_expr
```

Logische Operatoren

Basisprädikate
und -ausdrücke

Berechnungen

Einschränken von Ergebnismengen

■ Datums- und Zeitvergleiche sowie -berechnungen

- Eine große Stärke von Datenbanken ist das Handling von Datumswerten.
 - Vgl. <https://dev.mysql.com/doc/refman/5.7/en/date-calculations.html>
- Einfache Zeitvergleiche basieren auch auf „literals“ in „simple expressions“.
 - Folgendes Format ist für „timestamp“-Spalten empfohlen:

```
select * from produkt  
where angelegt_am >= '2020-11-24 13:30:00'
```

- Es sind komplexe Berechnungen und Vergleiche möglich.
 - Vgl. <https://dev.mysql.com/doc/refman/5.7/en/date-and-time-functions.html>
 - Vergleiche bei Zeitstempeln führen zu „Intervallen“, die in beliebigen Einheiten angegeben werden können.
 - Beispiel: vor wie vielen Tagen wurde ein Produkt angelegt.

```
select bezeichnung, timestampdiff(day, angelegt_am, curdate()) as alter_in_tagen  
from produkt;
```
 - Beispiel: wie viele Tage vor Weihnachten wurde eine Bestellung erzeugt:

```
select bestnr, datediff('2020-12-24', bestelldatum) as tage_bis_heiligabend  
from bestellung;
```


Einschränken von Ergebnismengen

■ Datums- und Zeitvergleiche sowie -berechnungen (cont'd)

- Diese Berechnungen können dementsprechend in WHERE-Bedingungen verwendet werden.

- Beispiel: alle Bestellungen, die maximal 30 Tage sind:

```
select * from bestellung
where datediff(bestelldatum, curdate()) >= -30
```

- Beispiel: alle Bestellungen, die an Montagen eingegangen sind:

```
select *
from bestellung
where dayname(bestelldatum) = 'Monday'
```

- Vgl. <https://dev.mysql.com/doc/refman/5.7/en/date-calculations.html>

Einschränken von Ergebnismengen

- Die Datumswertberechnungen sind ein Beispiel für die Verwendung von **Funktionen** (*function_call*) im WHERE-Statement

```
expr:
  expr OR expr
| expr || expr
| expr XOR expr
| expr AND expr
| expr && expr
| NOT expr
| ! expr
| boolean_primary IS [NOT] {TRUE | FALSE | UNKNOWN}
| boolean_primary

boolean_primary:
  boolean_primary IS [NOT] NULL
| boolean_primary <=> predicate
| boolean_primary comparison_operator predicate
| boolean_primary comparison_operator {ALL | ANY} (subquery)
| predicate

comparison_operator: = | >= | > | <= | < | <> | !=

predicate:
  bit_expr [NOT] IN (subquery)
| bit_expr [NOT] IN (expr [, expr] ...)
| bit_expr [NOT] BETWEEN bit_expr AND predicate
| bit_expr SOUNDS LIKE bit_expr
| bit_expr [NOT] LIKE simple_expr [ESCAPE simple_expr]
| bit_expr [NOT] REGEXP bit_expr
| bit_expr

bit_expr:
  bit_expr | bit_expr
| bit_expr & bit_expr
| bit_expr << bit_expr
| bit_expr >> bit_expr
| bit_expr + bit_expr
| bit_expr - bit_expr
| bit_expr * bit_expr
| bit_expr / bit_expr
| bit_expr DIV bit_expr
| bit_expr MOD bit_expr
| bit_expr % bit_expr
| bit_expr ^ bit_expr
| bit_expr + interval_expr
| bit_expr - interval_expr
| simple_expr

simple_expr:
  literal
| identifier
| function_call
| simple_expr COLLATE collation_name
| param_marker
| variable
| simple_expr || simple_expr
| + simple_expr
| - simple_expr
| ~ simple_expr
| ! simple_expr
| BINARY simple_expr
| (expr [, expr] ...)
| ROW (expr, expr [, expr] ...)
| (subquery)
| EXISTS (subquery)
| {identifier expr}
| match_expr
| case_expr
| interval_expr
```

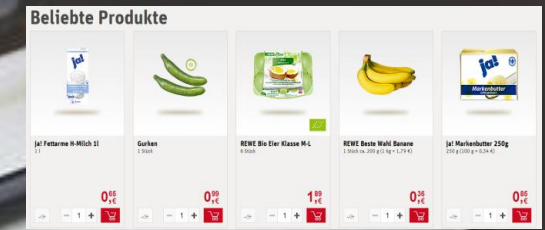
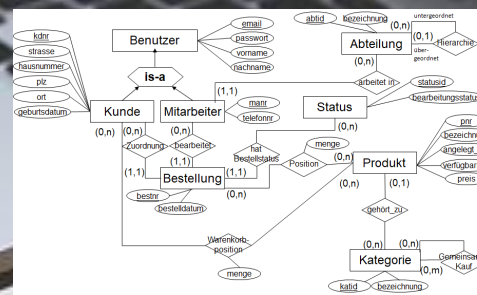
Logische Operatoren

Basisprädikate und -ausdrücke

Berechnungen

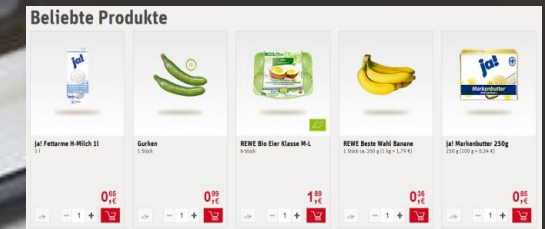
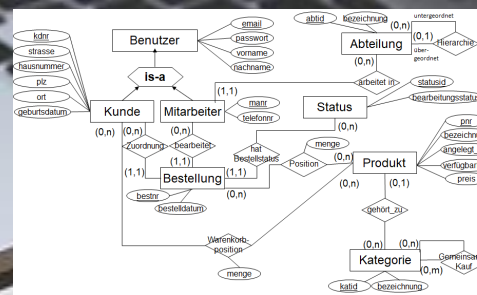
Funktionsaufrufe

Fallstudie



- DQL 2.1 ■ Selektieren Sie alle Bestellungen, deren Statusid 1, 3 oder 5 lautet und die ab dem '2024-10-23 00:00:00' eingegangen sind.
- DQL 2.2 ■ Selektieren Sie alle Produkte, denen keine Kategorie zugewiesen wurde.
- DQL 2.3 ■ Selektieren Sie alle Bestellpositionen, bei denen die Bestellmenge im Bereich zwischen 20 und 30 Stück (beide inklusive) liegt.
- DQL 2.4 ■ Selektieren Sie alle Benutzer, die eine Emailadresse mit der Domain "ab.de" angegeben haben.

Fallstudie



DQL 2.5 ■

In der Absatzprognose mit Machine Learning kann die Zeit bis zu einem bestimmten Ereignis sehr relevant sein. Wir analysieren dies nun zunächst deskriptiv: Selektieren Sie alle Bestellungen, bei denen die Tage bis Silvester 2023 weniger als 60 waren. Nutzen Sie die Funktion datediff. Zeigen Sie bei der Ausgabe zusätzlich zu allen Spalten auch die Tage bis Silvester der Bestellungen an.

DQL 2.6 ■

Selektieren Sie alle Bestellungen, die nachts zwischen 22 Uhr und 4 Uhr morgens getätigt wurden.

- Die Stunde eines Timestamp-Felds können Sie mit dem Befehl „`HOUR(spaltenname)`“ ermitteln.
- Vgl. https://dev.mysql.com/doc/refman/5.7/en/date-and-time-functions.html#function_hour

Agenda

- ☐ Query-Struktur und einfache Abfragen
- ☐ WHERE-Ausdrücke
- ☒ Gruppierung und Aggregation
- ☐ Kombination von Tabellen: JOIN und UNION
- ☐ Subqueries und Common Table Expressions
- ☐ Zusammenfassung und Ausblick

Aggregationsfunktionen

Die anzuzeigenden Aggregationen werden in der Spaltenliste formuliert.

```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

Gruppierung und
Einschränkungen auf Gruppen

- Aggregation und Gruppierung von Daten sind Grundlage zahlreicher Auswertungen, Berichte und Dashboards.
 - Beispiele: Gesamtumsatz, Durchschnittsumsatz pro Tag, Maximalumsatz pro Kategorie, ...
- SQL erlaubt die Anwendung vieler **Aggregationsfunktionen**
 - Beispiele: Anzahl, Summe, Durchschnitt, Minimum, Maximum, ...
- Diese können mit der **Gruppierung** von Daten kombiniert werden.
 - Gruppierungen können auch mehrstufig sein (bspw. Jahr → Produktgruppe)
 - Analog zum WHERE können mittels HAVING Einschränkungen auf Gruppenebene definiert werden (bspw. nur Kategorien mit mindestens drei Produkten).

Aggregationsfunktionen

■ MySQL-Aggregationsfunktionen

Name	Description
<u>AVG ()</u>	Return the average value of the argument
<u>BIT AND ()</u>	Return bitwise AND
<u>BIT OR ()</u>	Return bitwise OR
<u>BIT XOR ()</u>	Return bitwise XOR
<u>COUNT ()</u>	Return a count of the number of rows returned
<u>COUNT (DISTINCT)</u>	Return the count of a number of different values
<u>GROUP CONCAT ()</u>	Return a concatenated string
<u>JSON_ARRAYAGG ()</u> (introduced 5.7.22)	Return result set as a single JSON array
<u>JSON_OBJECTAGG ()</u> (introduced 5.7.22)	Return result set as a single JSON object
<u>MAX ()</u>	Return the maximum value
<u>MIN ()</u>	Return the minimum value
<u>STD ()</u>	Return the population standard deviation
<u>STDDEV ()</u>	Return the population standard deviation
<u>STDDEV_POP ()</u>	Return the population standard deviation
<u>STDDEV_SAMP ()</u>	Return the sample standard deviation
<u>SUM ()</u>	Return the sum
<u>VAR_POP ()</u>	Return the population standard variance
<u>VAR_SAMP ()</u>	Return the sample variance
<u>VARIANCE ()</u>	Return the population standard variance

Für uns relevante Aggregatsfunktionen.

Aggregationsfunktionen

■ Grundlagen zu Aggregatsfunktionen

- Aggregatsfunktionen erwarten ein **Argument** → fast immer ein **Spaltenname!**
 - Berechne den Durchschnittspreis → Preis ist das Argument für avg → avg(**preis**)
- Wird eine Aggregatsfunktion ohne GROUP BY (s.u.) verwendet, dann werden alle Zeilen der Tabelle zur Berechnung herangezogen.
 - Ausnahme: NULL-Werte (s.u.)

Inhalt Produkttabelle:

(Spalten pnr, bezeichnung und preis)

123 pnr	ABC bezeichnung	123 preis
1	Schokoladenpudding	0,8
2	Eier (Freilandhaltung)	2,2
3	Banane	0,23
4	Zitrone	0,34
5	Blumenkohl	1,8
6	Grillwurst	3,6
7	Weißer Schokolade	1,1
8	Zartbitterschokolade	0,9
9	Vollmilch	0,9
10	Katzenfutter	2,2
11	Hundefutter	2,4

Beispiele:

```
select count(pnr) as  
anzahl_produkte  
from produkt
```

	123 anzahl_produkte
1	11

```
select avg(preis) as  
durchschnittspreis  
from produkt
```

	123 durchschnittspreis
1	1,497273

Aggregationsfunktionen

■ Grundlagen zu Aggregatsfunktionen (cont'd)

- NULL-Werte werden grundsätzlich **ignoriert** (falls nicht anders angegeben)

Inhalt Produkttabelle:

123 pnr	ABC bezeichnung	123 katid
1	Schokoladenpudding	1
2	Eier (Freilandhaltung)	2
3	Banane	4
4	Zitrone	4
5	Blumenkohl	5
6	Grillwurst	6
7	Weißer Schokolade	3
8	Zartbitterschokolade	3
9	Vollmilch	1
10	Katzenfutter	[NULL]
11	Hundefutter	[NULL]

```
select count(katid) as  
anzahl_produkte_mit_kategoriezuordnung  
from produkt
```

	123 anzahl_produkte_mit_kategoriezuordnung
1	9

Aggregationsfunktionen

■ Grundlagen zu Aggregatsfunktionen (cont'd)

- Bei der Aggregatsfunktion „count“ kann als Argument * übergeben werden.
 - Interpretation: zähle alle Zeilen einer Tabelle, bei denen in mindestens einer Spalte kein NULL-Wert eingetragen ist

Inhalt Produkttabelle:

123 pnr	ABC bezeichnung	123 katid
1	Schokoladenpudding	1
2	Eier (Freilandhaltung)	2
3	Banane	4
4	Zitrone	4
5	Blumenkohl	5
6	Grillwurst	6
7	Weißer Schokolade	3
8	Zartbitterschokolade	3
9	Vollmilch	1
10	Katzenfutter	[NULL]
11	Hundefutter	[NULL]

```
select count(*) as anzahl_produkte  
from produkt
```

	123 anzahl_produkte
1	11

Aggregationsfunktionen

■ Grundlagen zu Aggregatsfunktionen (cont'd)

- Die Aggregatsfunktion „count“ kann mit **DISTINCT** kombiniert werden.
→ Interpretation: ermittle die Anzahl eindeutiger Werte in einer Spalte

Inhalt Produkttabelle:

123 pnr	ABC bezeichnung	123 katid
1	Schokoladenpudding	1
2	Eier (Freilandhaltung)	2
3	Banane	4
4	Zitrone	4
5	Blumenkohl	5
6	Grillwurst	6
7	Weißer Schokolade	3
8	Zartbitterschokolade	3
9	Vollmilch	1
10	Katzenfutter	[NULL]
11	Hundefutter	[NULL]

```
select count(distinct katid) as  
anzahl_referenzierter_kategorien  
from produkt
```

	123 anzahl_referenzierter_kategorien
1	6

Aggregationsfunktionen

■ Grundlagen zu Aggregatsfunktionen (cont'd)

- Alle Aggregatsfunktionen können mit **WHERE** kombiniert werden.
 - Interpretation: wende die Aggregatsfunktion nur auf Zeilen an, die der WHERE-Bedingung entsprechen

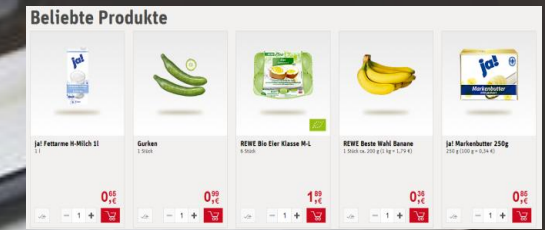
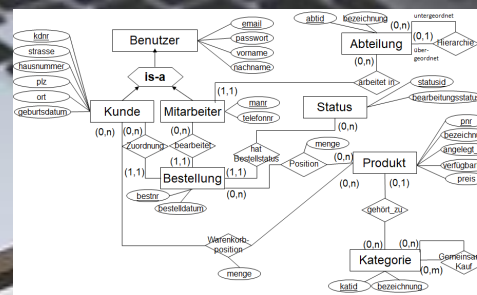
Inhalt Produkttabelle:

123 pnr 🔍 ⚙	ABC bezeichnung 🔍 ⚙	123 preis 🔍 ⚙
1	Schokoladenpudding	0,8
2	Eier (Freilandhaltung)	2,2
3	Banane	0,23
4	Zitrone	0,34
5	Blumenkohl	1,8
6	Grillwurst	3,6
7	Weißer Schokolade	1,1
8	Zartbitterschokolade	0,9
9	Vollmilch	0,9
10	Katzenfutter	2,2
11	Hundefutter	2,4

```
select count(*) as  
anzahl_produkte_ueber_3euro  
from produkt  
where preis > 3
```

	123 anzahl_produkte_ueber_3euro 🔍 ⚙
1	1

Fallstudie



- DQL 3.1** ■ Geben Sie die Anzahl an Produkten, den Durchschnittsbestand (Bestand = Verfügbarkeit), sowie den Maximal- und Minimalbestand aller Produkte aus.
- Vergeben Sie gut interpretierbare Aliase für die berechneten Werte.
- DQL 3.2** ■ Geben Sie die Anzahl an Abteilungen aus, die unter einer anderen Abteilung in der Hierarchie angeordnet sind.
- DQL 3.3** ■ Geben Sie die Anzahl von eindeutigen Orten aus, die in der Kundentabelle zu finden ist.
- DQL 3.4** ■ Geben Sie den durchschnittlichen Lagerbestandswert (preis*verfuegbarkeit) der Produkte aus, die zu Kategorien 3-6 gehören.

Gruppierung – Grundlagen

Empfehlung: die Gruppierungsspalte(n)
vor den Aggregatsfunktionen anzeigen.

```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

Gruppierung und
Einschränkungen auf Gruppen

- Aggregatsfunktionen in Kombination mit Gruppierung
→ Ermittlung der Berechnungsergebnisse (Aggregationen) pro Gruppe

Inhalt Kundentabelle:

ABC email	ABC ort
hans.mueller@xy.de	Essen
lea.schmidt@xy.de	Karlsruhe
leonie.hausmann@ms.de	Münster
peter.schulz@ab.de	Karlsruhe

(1) Ermittlung der „distinkten“ Werte
der Gruppierungsspalte(n).

Empfohlen: Gruppierungsspalte(n) anzeigen

```
select ort, count(email) as anzahl_kunden
from kunde
group by ort
```

ABC ort	123 anzahl_kunden
Essen	1
Karlsruhe	2
Münster	1

(2) Aggregation je
distinktem Wert der
Gruppierungsspalte(n)

Gruppierung

■ Grundlagen zu Gruppierung

- Gruppierung & Aggregation kann mit **WHERE** kombiniert werden.
 - Interpretation: nimm in die Gruppierung nur Zeilen auf, die der WHERE-Bedingung entsprechen. Gruppiere danach die Daten und wende die Aggregatsfunktion an.
- *Hinweis: Eine Kombination mit ORDER BY ist auch ohne Weiteres möglich!*

Inhalt Produkttabelle:

123 pnr	ABC bezeichnung	123 preis	123 katid
1	Schokoladenpudding	0,8	1
2	Eier (Freilandhaltung)	2,2	2
3	Banane	0,23	4
4	Zitrone	0,34	4
5	Blumenkohl	1,8	5
6	Grillwurst	3,6	6
7	Weißer Schokolade	1,1	3
8	Zartbitterschokolade	0,9	3
9	Vollmilch	0,9	1
10	Katzenfutter	2,2	[NULL]
11	Hundefutter	2,4	[NULL]

```
select katid, count(pnr) as  
anzahl_produkte_ueber_1euro  
from produkt  
where preis > 1  
group by katid
```

123 katid	123 anzahl_produkte_ueber_1euro
[NULL]	2
2	1
3	1
5	1
6	1

Gruppierung

■ Grundlagen zu Gruppierung (cont'd)

- Das Gruppierungsattribut darf ein **berechneter Wert / Funktionsaufruf** sein.
 - Beispiel: gruppieren nach dem „auf den vollen Euro gerundeten Preis“ und gib die Anzahl Produkte jeweils aus.

Inhalt Produkttabelle mit berechneter Spalte:

```
select pnr, bezeichnung, preis, round(preis) as  
gerundeter_preis from produkt
```

123 pnr	ABC bezeichnung	123 preis	123 gerundeter_preis
1	Schokoladenpudding	0,8	1
2	Eier (Freilandhaltung)	2,2	2
3	Banane	0,23	0
4	Zitrone	0,34	0
5	Blumenkohl	1,8	2
6	Grillwurst	3,6	4
7	Weißer Schokolade	1,1	1
8	Zartbitterschokolade	0,9	1
9	Vollmilch	0,9	1
10	Katzenfutter	2,2	2
11	Hundefutter	2,4	2

```
select round(preis) as gerundeter_preis,  
count(pnr) as anzahl_produkte  
from produkt  
group by gerundeter_preis
```

123 gerundeter_preis	123 anzahl_produkte
0	2
1	4
2	4
4	1

Gruppierung

■ Grundlagen zu Gruppierung (cont'd)

- Es kann nach **mehr als einem Attribut** gruppiert werden.
 - Auch in dem Fall empfiehlt es sich, alle Gruppierungsspalten ausgeben zu lassen.
 - Oftmals soll mehr als ein Attribut pro Gruppe angezeigt werden (bspw. Produktnummer und Produktbezeichnung) → in dem Fall müssen beide Attribute hinter GROUP BY auftauchen

Inhalt Produkttabelle mit berechneter Spalte:

```
select pnr, bezeichnung, preis, round(preis) as  
gerundeter_preis, katid from produkt
```

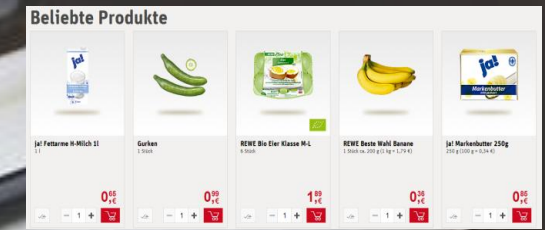
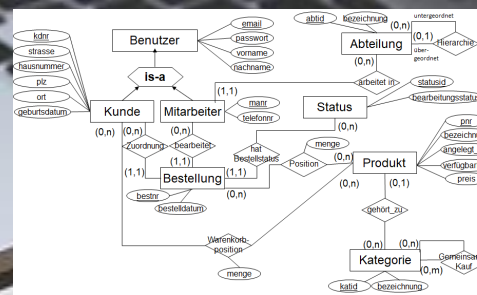
123 pnr	ABC bezeichnung	123 preis	123 gerundeter_preis	123 katid
1	Schokoladenpudding	0,8	1	1
2	Eier (Freilandhaltung)	2,2	2	2
3	Banane	0,23	0	4
4	Zitrone	0,34	0	4
5	Blumenkohl	1,8	2	5
6	Grillwurst	3,6	4	6
7	Weißer Schokolade	1,1	1	3
8	Zartbitterschokolade	0,9	1	3
9	Vollmilch	0,9	1	1
10	Katzenfutter	2,2	2	[NULL]
11	Hundefutter	2,4	2	[NULL]

Gruppierung nach
eindeutigen Kombinationen
beider Spalten

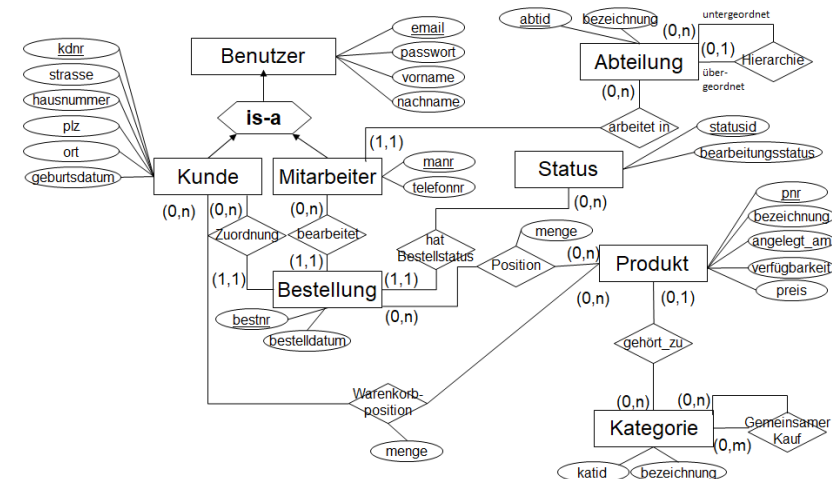
```
select round(preis) as gerundeter_preis,  
katid, count(pnr) as anzahl_produkte  
from produkt  
group by gerundeter_preis, katid
```

123 gerundeter_preis	123 katid	123 anzahl_produkte
0	4	2
1	1	2
1	3	2
2	[NULL]	2
2	2	1
2	5	1
4	6	1

Fallstudie



- DQL 3.5 ■** Geben Sie die Anzahl Bestellungen (danach absteigend sortiert) je Kunde aus.
- DQL 3.6 ■** Geben Sie je Kunde und StatusID (Join nicht nötig!) die Anzahl Bestellungen aus. Beschränken Sie die Analyse nur auf die StatusIDs 1, 3 und 5.
- DQL 3.7 ■** Wir möchten eine Top3-Analyse der am meisten verkauften Produkte (Feld pnr genügt, Join unnötig!) erstellen. Welche Tabelle müssen wir analysieren? Erzeugen Sie ein entsprechendes SQL-Statement.



Gruppierung

■ Hinweis zum Gruppierungsattribut

- Das Gruppierungsattribut muss nicht zwingend in der SELECT-Spaltenliste vor der/den Aggregatsfunktion(en) angegeben werden.

123  katid 	ABC bezeichnung 	123  anzahl_produkte 
1	Molkereiprodukte	2
2	Eier	1
3	Süßwaren	2
4	Obst	2
5	Gemüse	1
6	Fleisch	1
7	Fisch	1

Hinweis: Bevor wir dieses Beispiel in SQL umsetzen können, müssen wir die Joins (s.u.) kennenlernen.

Eine mögliche Anforderung:

Gruppierung nach der eindeutigen ID-Spalte – diese soll aber **nicht** angezeigt werden (stattdessen nur die Bezeichnung)

- Wichtig: dies funktioniert nur, wenn die anzuzeigende Spalte „funktional abhängig“ von der Gruppierungsspalte ist.
 - Im Beispiel: Bezeichnung ist funktional abhängig von katid.

Gruppierung – Bedingungen auf Gruppen

```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

Gruppierung und
Einschränkungen auf Gruppen

- Das HAVING-Attribut lässt Einschränkungen (=Bedingungen) auf Gruppenebene zu
 - Wichtig: dies ist etwas anderes als die WHERE-Bedingung!
 - WHERE schränkt die Zeilen ein, die überhaupt in die Gruppierung eingehen („arbeitet“ also VOR der Gruppierung).
 - Beispiel: nur Produkte in die Gruppierung aufnehmen, deren Preis > 1 Euro ist
 - HAVING schränkt die Gruppen ein, die vom SELECT-Statement ausgegeben werden („arbeitet“ also NACH der Gruppierung)
 - Beispiel: nur Gruppen anzeigen, in denen mindestens 5 Produkte vorkommen.

Gruppierung – Bedingungen auf Gruppen

■ Bedingungen auf Gruppenebene

- MySQL erlaubt die Verwendung von Aliassen bei HAVING
 - Beispiel: gib den Durchschnittspreis je Kategorie aus, jedoch nur für Kategorien, denen mindestens zwei Produkte (anzahl_produkte) zugeordnet sind.
 - Bei PostgreSQL müsste das Statement „having count(pnr) >= 2“ lauten!
 - Auch wenn kein Alias bzw. die Aggregatsfunktion nicht in der Spaltenausgabe verwendet wird muss die Aggregatsfunktion im HAVING-Teil spezifiziert werden!

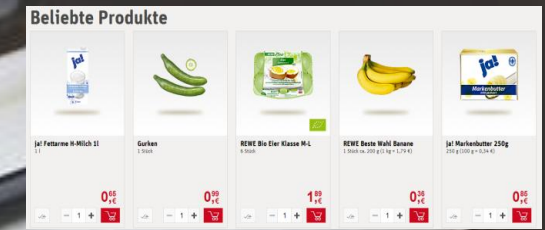
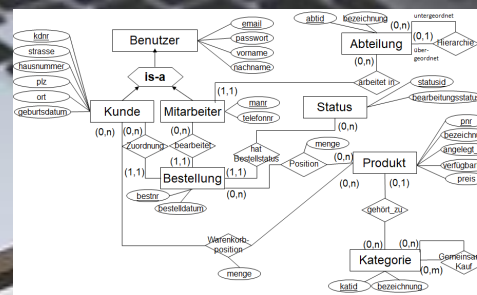
Inhalt Produkttabelle:

123 pnr	ABC bezeichnung	123 katid	123 preis
1	Schokoladenpudding	1	0,8
2	Eier (Freilandhaltung)	2	2,2
3	Banane	4	0,23
4	Zitrone	4	0,34
5	Blumenkohl	5	1,8
6	Grillwurst	6	3,6
7	Weißer Schokolade	3	1,1
8	Zartbitterschokolade	3	0,9
9	Vollmilch	1	0,9
10	Katzenfutter	[NULL]	2,2
11	Hundefutter	[NULL]	2,4

```
select katid, count(pnr) as  
anzahl_produkte, avg(preis) as  
durchschnittspreis  
from produkt  
group by katid  
having anzahl_produkte >= 2
```

123 katid	123 anzahl_produkte	123 durchschnittspreis
[NULL]	2	2,3
1	2	0,85
3	2	1
4	2	0,285

Fallstudie



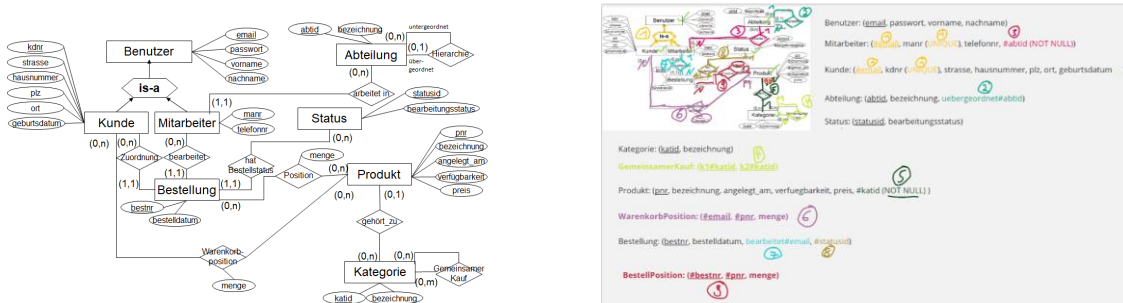
- DQL 3.8 ■** Geben Sie je Produktnummer die durchschnittliche Bestellmenge (Tabelle Bestellposition) gerundet auf zwei Stellen aus.
- Runden können Sie mit `round(spaltenname, anzahl_stellen)` bzw. bei Aggregatsfunktionen mit `round(aggregatsfunktion(spaltenname), anzahl_stellen)`
 - Vgl. https://dev.mysql.com/doc/refman/5.7/en/mathematical-functions.html#function_round
- DQL 3.9 ■** Geben Sie je Produktnummer die Anzahl an Bestellpositionen aus; Sortieren Sie absteigend nach Anzahl an Bestellpositionen.
- Merken Sie sich die Anzahl an Bestellpositionen in der ersten Zeile - 10.
- DQL 3.10 ■** Geben Sie je Produktnummer die durchschnittliche Bestellmenge (Tabelle Bestellposition) gerundet auf zwei Stellen aus. Geben Sie nur die Produktnummern aus, bei denen mindestens genauso viele Bestellpositionen existieren wie die Zahl, die Sie sich gemerkt haben.

Agenda

- ☐ Query-Struktur und einfache Abfragen
- ☐ WHERE-Ausdrücke
- ☐ Gruppierung und Aggregation
- ☒ Kombination von Tabellen: JOIN und UNION
- ☐ Subqueries und Common Table Expressions
- ☐ Zusammenfassung und Ausblick

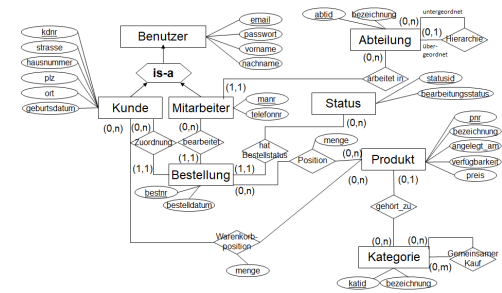
Joins in Datenbanken – Grundlagen

- Das relationale Modell vermeidet redundante Datenspeicherung
→ **Normalisierung** / normalisierte Datenmodelle
 - Tabellen einer Datenbank sind über Fremdschlüsselattribute miteinander verknüpft.



- Für Auswertungszwecke und aussagekräftige Reports müssen die normalisierten Daten oftmals wieder **zusammengeführt** werden.
 - *Beispiel: Anzahl Produkte pro Kategorie*
→ die katid ist in der Tabelle Produkt gespeichert, aber eigentlich möchten wir die Kategoriebezeichnung ausgeben lassen.
 - Mittels Joins kann SELECT-Abfragen durch Angabe einer „Suchbedingung“ die (Fremdschlüssel-)verknüpfung zwischen Tabellen explizit genutzt werden.
→ Die Verknüpfung wird nur für die Abfrage hergestellt (keine Änderung am Tabellenschema!)

Joins in Datenbanken – Grundlagen



- Wir können mehrere Tabellen hinter „FROM“ angeben.

```
SELECT [DISTINCT] spaltenliste
FROM tabellenliste
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

- Dies führt jedoch ggf. nicht ohne Weiteres zum gewünschten Ergebnis.

```
select *
from produkt, kategorie
```

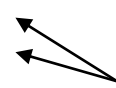
- Jede Zeile von Tabelle 1 (*produkt*) wird mit jeder Zeile aus Tabelle 2 (*kategorie*) kombiniert.
 → Kartesisches Produkt
 → = CROSS JOIN

	pnr	ABC bezeichnung	angelegt_am	123 veruegbarkeit	123 preis	123 katid	katid	ABC bezeichnung
1	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	1	Molkereiprodukte
2	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	2	Eier
3	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	3	Süßwaren
4	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	4	Obst
5	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	5	Gemüse
6	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	6	Fleisch
7	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	7	Fisch
8	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	1	Molkereiprodukte
9	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	2	Eier
10	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	3	Süßwaren
11	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	4	Obst
12	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	5	Gemüse
13	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	6	Fleisch
14	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	7	Fisch
15	3	Banane	2020-11-24 14:10:55.0	150	0,23	4	1	Molkereiprodukte
16	3	Banane	2020-11-24 14:10:55.0	150	0,23	4	2	Eier
17	3	Banane	2020-11-24 14:10:55.0	150	0,23	4	3	Süßwaren
18	3	Banane	2020-11-24 14:10:55.0	150	0,23	4	4	Obst
...	Spalten aus Tabelle Produkt						Spalten aus Tabelle Kategorie	

Joins in Datenbanken – Grundlagen

- Um das implizite Verhalten zu vermeiden wird die explizite Verknüpfung von zwei oder mehr Tabellen mittels JOIN empfohlen.

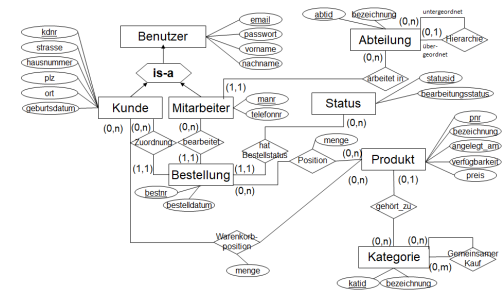
```
SELECT [DISTINCT] spaltenliste
FROM tabelle1 t1
    [INNER | CROSS] | [{LEFT|RIGHT}] [OUTER] JOIN tabelle2 t2 ...
[ [INNER | CROSS] | [{LEFT|RIGHT}] [OUTER] ] JOIN tabelle3 t2 ...
[ ... ]
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```



Hier werden wir die Attribute aus beiden Tabellen, über die die Verknüpfung hergestellt werden soll, spezifizieren (üblicherweise Fremd- und Primärschlüssel, bspw. produkt.katid, kategorie.katid)

- Mittels CROSS-Join kann das kartesische Produkt erzeugt werden (selten nötig!)
- Am häufigsten verwendete Join-Typen sind **INNER-** sowie **LEFT-Joins**.
- In der Spaltenliste können Spalten aus beiden Tabellen selektiert werden.
- Tabellenalias (bspw. *t1*, *t2*) sind empfohlen, um die Lesbarkeit zu erhöhen.

Joins in Datenbanken – Grundlagen



- Durch den Join (außer Cross-Join) sollen nur die Zeilen zurückgegeben werden, bei denen die „verknüpfenden“ Attribute (Join-Spalten) denselben Wert haben.

Ohne Join:

```
select *
from produkt, kategorie
```

Mit Join:

```
select *
from produkt p
join kategorie k using (katid)
```

Alternative (insb. bei unterschiedlichen Spaltennamen):

```
select *
from produkt p
join kategorie k on p.katid = k.katid
```

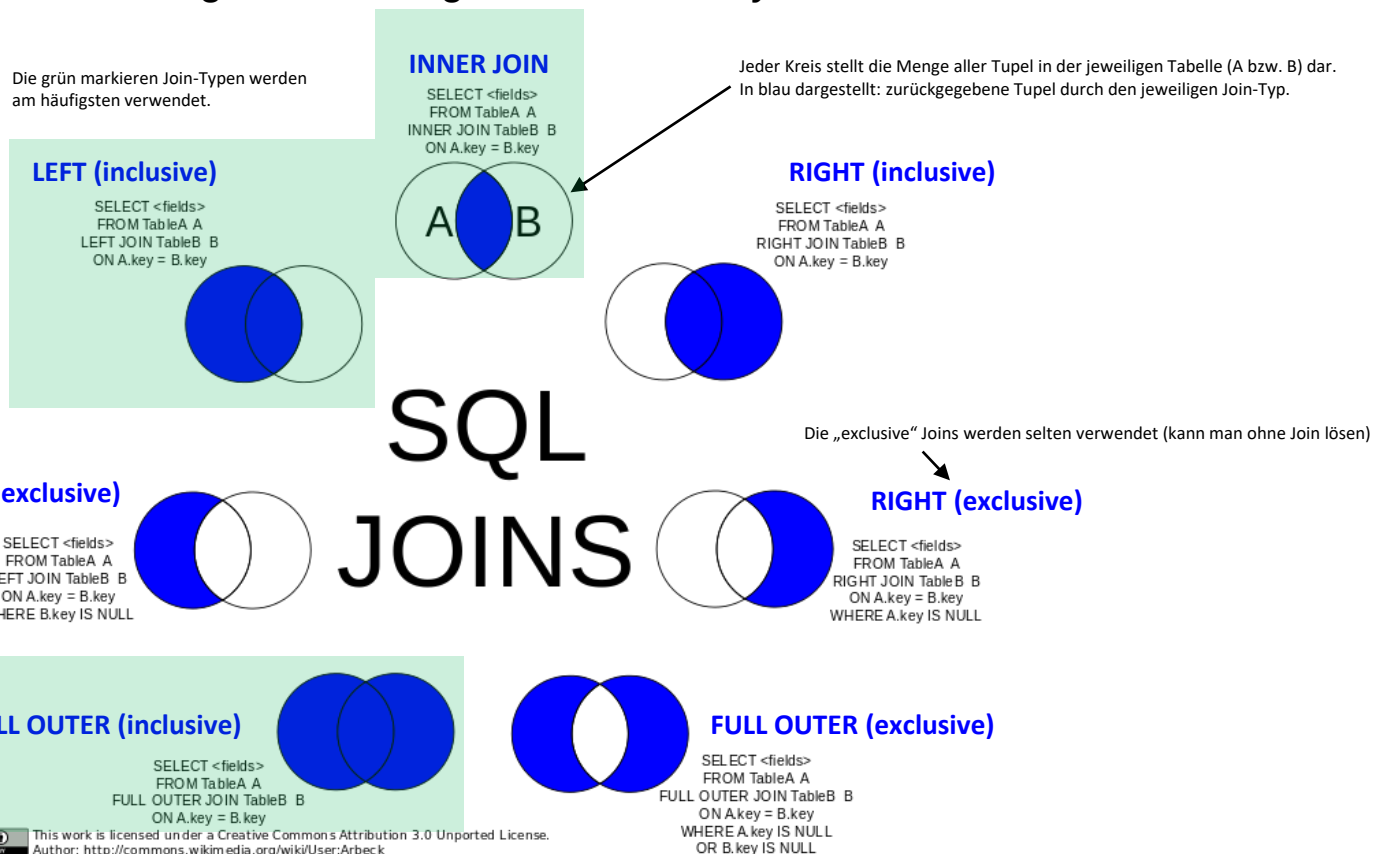
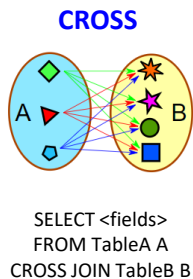
Join-Spalten									
123 pnr	ABC bezeichnung	angelegt_am	123 veruegbarkeit	123 preis	123 katid	=	123 katid	ABC bezeichnung	
1	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	1	Molkereiprodukte	
2	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	2	Eier	
3	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	3	Süßwaren	
4	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	4	Obst	
5	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	5	Gemüse	
6	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	6	Fleisch	
7	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1	=	7	Fisch	
8	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	1	Molkereiprodukte	
9	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	2	Eier	
10	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	3	Süßwaren	
11	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	4	Obst	
12	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	5	Gemüse	
13	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	6	Fleisch	
14	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2	=	7	Fisch	
15	3 Banane	2020-11-24 14:10:55.0	150	0,23	4	=	1	Molkereiprodukte	
16	3 Banane	2020-11-24 14:10:55.0	150	0,23	4	=	2	Eier	
17	3 Banane	2020-11-24 14:10:55.0	150	0,23	4	=	3	Süßwaren	
18	3 Banane	2020-11-24 14:10:55.0	150	0,23	4	=	4	Obst	

Spalten aus Tabelle Produkt									
123 katid	123 pnr	ABC bezeichnung	angelegt_am	123 veruegbarkeit	123 preis	Spalten aus Tabelle Kategorie			
1	1	1 Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	Molkereiprodukte			
2	1	9 Vollmilch	2020-09-24 13:13:23.0	80	0,9	Molkereiprodukte			
3	2	2 Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	Eier			
4	3	7 Weiße Schokolade	2020-12-01 09:15:59.0	90	1,1	Süßwaren			
5	3	8 Zartbitterschokolade	2020-11-28 16:15:59.0	106	0,9	Süßwaren			
6	4	3 Banane	2020-11-24 14:10:55.0	150	0,23	Obst			
7	4	4 Zitrone	2020-11-25 11:13:59.0	80	0,34	Obst			
8	5	5 Blumenkohl	2020-10-24 12:15:59.0	10	1,8	Gemüse			
9	6	6 Grillwurst	2020-10-24 12:14:48.0	50	3,6	Fleisch			

Join-Spalte mit demselben Namen
in beiden Tabellen

Joins in Datenbanken – Grundlagen

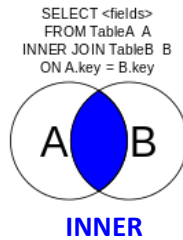
- Durch die verschiedenen **Join-Typen** werden unterschiedliche Anforderungen umgesetzt.
 - Abhängig von der Aufgabenstellung bzw. dem Analyseziel.



Quellen: Arbeck, CC BY 3.0 <<https://creativecommons.org/licenses/by/3.0/>>, via Wikimedia Commons, GermanX, CC BY-SA 4.0 <<https://creativecommons.org/licenses/by-sa/4.0/>>, via Wikimedia Commons

DQL: Joins

Joins in Datenbanken – INNER JOIN



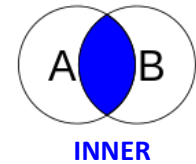
- Der INNER JOIN wird im Standard verwendet, wenn nichts anderes angegeben ist.
 - Eine explizite Angabe des Join-Typs wird jedoch immer empfohlen!
 - Beim INNER JOIN werden nur die Zeilen zurückgegeben, für die eine Verknüpfung hergestellt werden kann.
- Jeder Join benötigt die Angabe der Spalten, die für die Verknüpfung in beiden Tabellen verwendet werden sollen.
 - Spaltennamen identisch: die Beziehung kann mit „USING“ spezifiziert werden

```
select *  
from produkt p  
inner join kategorie k using (katid)
```
 - Spaltennamen unterschiedlich: „ON“ mit einer „Suchbedingung“ verwenden

```
select *  
from bestellung b  
inner join kunde k on b.kunde = k.email
```

Joins in Datenbanken – INNER JOIN

```
SELECT <fields>
FROM TableA A
INNER JOIN TableB B
ON A.key = B.key
```



■ INNER JOIN

- Nur die Zeilen zurückgeben, für die die Verknüpfung zwischen den beiden Tabellen hergestellt werden kann.

Spaltenalias sind nötig, wenn in beiden Tabellen derselbe Spaltenname verwendet wird.

```
select p.pnr, p.bezeichnung as produkt,
       k.katid, k.bezeichnung as kategorie
from produkt p
inner join kategorie k on p.katid = k.katid;
```

Inhalt Tabelle Produkt:

	123 pnr	ABC bezeichnung	angelegt_am	123 veruegbarkeit	123 preis	123 katid
1	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1
2	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2
3	3	Banane	2020-11-24 14:10:55.0	150	0,23	4
4	4	Zitrone	2020-11-25 11:13:59.0	80	0,34	4
5	5	Blumenkohl	2020-10-24 12:15:59.0	10	1,8	5
6	6	Grillwurst	2020-10-24 12:14:48.0	50	3,6	6
7	7	Weißer Schokolade	2020-12-01 09:15:59.0	90	1,1	3
8	8	Zartbitterschokolade	2020-11-28 16:15:59.0	106	0,9	3
9	9	Vollmilch	2020-09-24 13:13:23.0	80	0,9	1
10	10	Katzenfutter	2020-11-30 12:17:42.0	10	2,2	[NULL]
11	11	Hundefutter	2020-11-30 12:17:48.0	18	2,4	[NULL]

Tauchen im Join-Ergebnis nicht auf
(beide Produkte haben keine katid zugewiesen)

123 pnr	ABC produkt	123 katid	ABC kategorie
1	Schokoladenpudding	1	Molkereiprodukte
9	Vollmilch	1	Molkereiprodukte
2	Eier (Freilandhaltung)	2	Eier
7	Weißer Schokolade	3	Süßwaren
8	Zartbitterschokolade	3	Süßwaren
3	Banane	4	Obst
4	Zitrone	4	Obst
5	Blumenkohl	5	Gemüse
6	Grillwurst	6	Fleisch

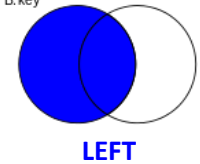
Inhalt Tabelle Kategorie:

123 katid	ABC bezeichnung
1	Molkereiprodukte
2	Eier
3	Süßwaren
4	Obst
5	Gemüse
6	Fleisch
7	Fisch

Taucht im Join-Ergebnis nicht auf
(kein Produkt hat katid 7 zugewiesen)

Joins in Datenbanken – LEFT JOIN

```
SELECT <fields>
FROM TableA A
LEFT JOIN TableB B
ON A.key = B.key
```



■ LEFT JOIN

- Alle Zeilen der linken Tabelle zurückgeben, auch wenn keine Verknüpfung zwischen den beiden Tabellen hergestellt werden kann.
- Wird eine Verknüpfung gefunden, dann werden die Inhalte aus der rechten Tabelle angezeigt (ansonsten NULL-Werte).

Die Tabelle vor dem Join-Statement ist die linke Tabelle. Die darauf folgende die rechte Tabelle.

```
select p.pnr, p.bezeichnung as produkt,
       k.katid, k.bezeichnung as kategorie
from produkt p
left join kategorie k on p.katid = k.katid;
```

Inhalt Tabelle Produkt:

	123 pnr	ABC bezeichnung	🕒 angelegt_am	123 verfügbarkeit	123 preis	123 katid
1	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1
2	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2
3	3	Banane	2020-11-24 14:10:55.0	150	0,23	4
4	4	Zitrone	2020-11-25 11:13:59.0	80	0,34	4
5	5	Blumenkohl	2020-10-24 12:15:59.0	10	1,8	5
6	6	Grillwurst	2020-10-24 12:14:48.0	50	3,6	6
7	7	Weißer Schokolade	2020-12-01 09:15:59.0	90	1,1	3
8	8	Zartbitterschokolade	2020-11-28 16:15:59.0	106	0,9	3
9	9	Vollmilch	2020-09-24 13:13:23.0	80	0,9	1
10	10	Katzenfutter	2020-11-30 12:17:42.0	10	2,2	[NULL]
11	11	Hundefutter	2020-11-30 12:17:48.0	18	2,4	[NULL]

Tauchen im Join-Ergebnis auch auf
(beide Produkte haben keine katid zugewiesen, aber LEFT JOIN)

123 pnr	ABC produkt	123 katid	ABC kategorie
1	Schokoladenpudding	1	Molkereiprodukte
2	Eier (Freilandhaltung)	2	Eier
3	Banane	4	Obst
4	Zitrone	4	Obst
5	Blumenkohl	5	Gemüse
6	Grillwurst	6	Fleisch
7	Weißer Schokolade	3	Süßwaren
8	Zartbitterschokolade	3	Süßwaren
9	Vollmilch	1	Molkereiprodukte
10	Katzenfutter	[NULL]	[NULL]
11	Hundefutter	[NULL]	[NULL]

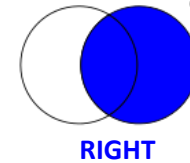
Inhalt Tabelle Kategorie:

123 katid	ABC bezeichnung
1	Molkereiprodukte
2	Eier
3	Süßwaren
4	Obst
5	Gemüse
6	Fleisch
7	Fisch

Taucht im Join-Ergebnis nicht auf
(kein Produkt hat katid 7 zugewiesen)

Joins in Datenbanken – RIGHT JOIN

```
SELECT <fields>
FROM TableA A
RIGHT JOIN TableB B
ON A.key = B.key
```



■ RIGHT JOIN

- Analog zum LEFT JOIN.
- LEFT JOIN ist intuitiver interpretierbar.
 - RIGHT JOIN kann durch „Umdrehen“ der Tabellenreihenfolge in einen LEFT JOIN umformuliert werden.

```
select p.pnr, p.bezeichnung as produkt,
       k.katid, k.bezeichnung as kategorie
from produkt p
right join kategorie k on p.katid = k.katid;
```

Inhalt Tabelle Produkt:

	123 pnr	ABC bezeichnung	🕒 angelegt_am	123 verfuegbarkeit	123 preis	123 katid
1	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1
2	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2
3	3	Banane	2020-11-24 14:10:55.0	150	0,23	4
4	4	Zitrone	2020-11-25 11:13:59.0	80	0,34	4
5	5	Blumenkohl	2020-10-24 12:15:59.0	10	1,8	5
6	6	Grillwurst	2020-10-24 12:14:48.0	50	3,6	6
7	7	Weißer Schokolade	2020-12-01 09:15:59.0	90	1,1	3
8	8	Zartbitterschokolade	2020-11-28 16:15:59.0	106	0,9	3
9	9	Vollmilch	2020-09-24 13:13:23.0	80	0,9	1
10	10	Katzenfutter	2020-11-30 12:17:42.0	10	2,2	[NULL]
11	11	Hundefutter	2020-11-30 12:17:48.0	18	2,4	[NULL]

Tauchen im Join-Ergebnis nicht auf
(beide Produkte haben keine katid zugewiesen)

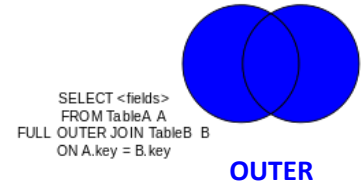
123 pnr	ABC produkt	123 katid	ABC kategorie
1	Schokoladenpudding	1	Molkereiprodukte
9	Vollmilch	1	Molkereiprodukte
2	Eier (Freilandhaltung)	2	Eier
7	Weißer Schokolade	3	Süßwaren
8	Zartbitterschokolade	3	Süßwaren
3	Banane	4	Obst
4	Zitrone	4	Obst
5	Blumenkohl	5	Gemüse
6	Grillwurst	6	Fleisch
[NULL]	[NULL]	7	Fisch

Inhalt Tabelle Kategorie:

	123 katid	ABC bezeichnung
1	1	Molkereiprodukte
2	2	Eier
3	3	Süßwaren
4	4	Obst
5	5	Gemüse
6	6	Fleisch
7	7	Fisch

Taucht im Join-Ergebnis auch auf
(kein Produkt hat katid 7 zugewiesen,
aber RIGHT JOIN)

Joins in Datenbanken – OUTER JOIN



■ (FULL) OUTER JOIN

- Alle Zeilen beider Tabellen, auch wenn keine Verknüpfung hergestellt werden kann.
 - Achtung: MySQL bietet den Befehl nicht (andere RDBMS setzen diesen um).
 - In MySQL können benötigen wir einen „Workaround“ mit UNION (s.u., hier nicht relevant).

```
select p.pnr, p.bezeichnung as produkt,
       k.katid, k.bezeichnung as kategorie
from produkt p
full outer join kategorie k on p.katid = k.katid;
```

Inhalt Tabelle Produkt:

	pnr	bezeichnung	angelegt_am	verfuegbarkeit	preis	katid
1	1	Schokoladenpudding	2020-11-24 12:15:59.0	100	0,8	1
2	2	Eier (Freilandhaltung)	2020-11-24 13:16:59.0	300	2,2	2
3	3	Banane	2020-11-24 14:10:55.0	150	0,23	4
4	4	Zitrone	2020-11-25 11:13:59.0	80	0,34	4
5	5	Blumenkohl	2020-10-24 12:15:59.0	10	1,8	5
6	6	Grillwurst	2020-10-24 12:14:48.0	50	3,6	6
7	7	Weißer Schokolade	2020-12-01 09:15:59.0	90	1,1	3
8	8	Zartbitterschokolade	2020-11-28 16:15:59.0	106	0,9	3
9	9	Vollmilch	2020-09-24 13:13:23.0	80	0,9	1
10	10	Katzenfutter	2020-11-30 12:17:42.0	10	2,2	[NULL]
11	11	Hundefutter	2020-11-30 12:17:48.0	18	2,4	[NULL]

Tauchen im Join-Ergebnis auch auf
(beide Produkte haben keine katid zugewiesen, aber OUTER JOIN)

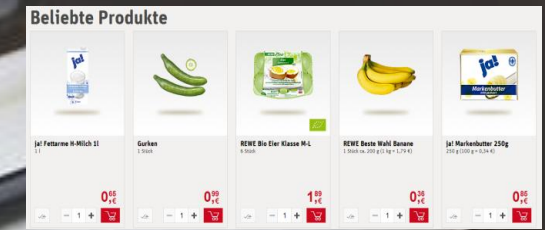
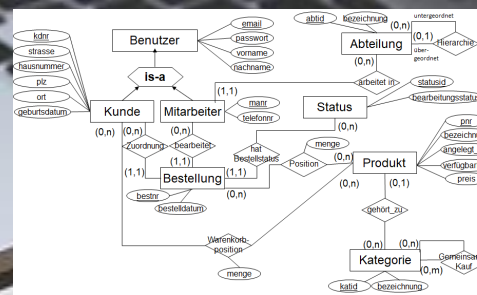
pnr	produkt	katid	kategorie
1	Schokoladenpudding	1	Molkereiprodukte
2	Eier (Freilandhaltung)	2	Eier
3	Banane	4	Obst
4	Zitrone	4	Obst
5	Blumenkohl	5	Gemüse
6	Grillwurst	6	Fleisch
7	Weißer Schokolade	3	Süßwaren
8	Zartbitterschokolade	3	Süßwaren
9	Vollmilch	1	Molkereiprodukte
10	Katzenfutter	[NULL]	[NULL]
11	Hundefutter	[NULL]	[NULL]
[NULL]	[NULL]	7	Fisch

Inhalt Tabelle Kategorie:

katid	bezeichnung
1	Molkereiprodukte
2	Eier
3	Süßwaren
4	Obst
5	Gemüse
6	Fleisch
7	Fisch

Taucht im Join-Ergebnis auch auf
(kein Produkt hat katid 7 zugewiesen,
aber OUTER JOIN)

Fallstudie



- DQL 4.1 ■** Geben Sie zu jeder Bestellung die Telefonnummer des bearbeitenden Mitarbeiters aus. Sortieren Sie absteigend nach bestellzeitpunkt.
- Auszugebende Spalten: bestnr, bestellzeitpunkt, telefonnr
- DQL 4.2 ■** Zeigen Sie zu jeder Bestellung (bestnr) den Status (Bezeichnung) an. Es sollen allerdings nur Bestellungen angezeigt werden, die nicht in der Statusbezeichnung „Zugestellt“ enthalten.
- DQL 4.3 ■** Zeigen Sie zu jedem Produkt (pnr) die zugeordnete Kategoriebezeichnung an. Ist keine Kategorie zugeordnet, soll NULL erscheinen.
- Nach welchem Attribut wird implizit beim Join sortiert?
- DQL 4.4 ■** Zeigen Sie für jede Kategorie (Bezeichnung) die Anzahl zugeordneter Produkte an. Sollten keine Produkte zugeordnet sein, so soll die Kategorie trotzdem erscheinen und 0 angezeigt werden.
- Gruppieren Sie nach der Spalte katid.
 - Auf welche Spalte sollte die Funktion „count“ angewendet werden?

Joins in Datenbanken – mehr als 2 Tabellen

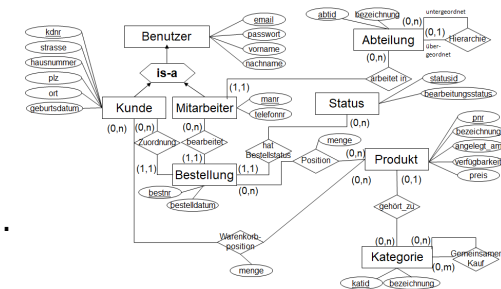
- Mit Joins können mehr als zwei Tabellen miteinander verbunden werden
 - Die Spalten und Zeilen aus dem ersten Join werden dann zur „linken Tabelle“ für den nächsten Join.
 - Es können unterschiedliche Join-Typen kombiniert werden.

```
SELECT [DISTINCT] spaltenliste
FROM tabelle1 t1
    [INNER | CROSS] | [{LEFT|RIGHT}] [OUTER] JOIN tabelle2 t2 ...
[ [INNER | CROSS] | [{LEFT|RIGHT}] [OUTER] ] JOIN tabelle3 t3 ...
[ ... ]
[WHERE bedingungsausdruck]
[GROUP BY spaltenliste]
[HAVING bedingungsausdruck]
[ORDER BY spaltenliste]
[LIMIT n]
```

- Auch DISTINCT und alle weiteren Statements, die wir kennengelernt haben, können verwendet werden (insbesondere Gruppierung und Aggregation).

Joins in Datenbanken – mehr als 2 Tabellen

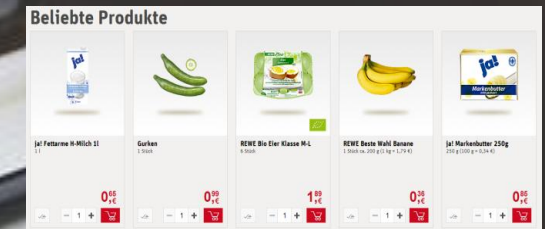
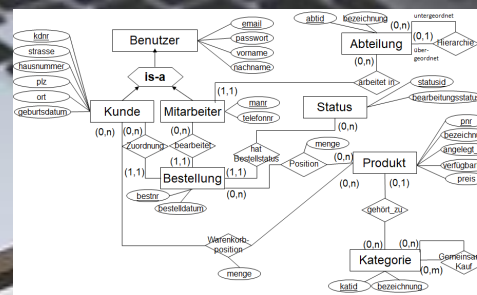
- Mit Joins können mehr als zwei Tabellen miteinander verbunden werden
 - Beispiel: für alle Kunden die Produktkategorien anzeigen, die im Warenkorb liegen.
 - Doppelte Werte sollen nicht angezeigt werden (DISTINCT!).
 - Ist das Produkt keiner Kategorie zugeordnet, so soll es trotzdem angezeigt werden
 - → An einer Stelle ist ein LEFT JOIN nötig!
 - Insgesamt müssen vier Tabellen miteinander verknüpft werden.



```
select distinct k.email, kat.bezeichnung as kategorie
from kunde k
inner join warenkorbposition wp on k.email = wp.kunde_email
inner join produkt p on wp.pnr = p.pnr
left join kategorie kat on p.katid = kat.katid
```

ABC email	ABC kategorie
peter.schulz@ab.de	Molkereiprodukte
hans.mueller@xy.de	Molkereiprodukte
lea.schmidt@xy.de	Molkereiprodukte
lea.schmidt@xy.de	Eier
lea.schmidt@xy.de	Obst
lea.schmidt@xy.de	Gemüse
lea.schmidt@xy.de	Fleisch
lea.schmidt@xy.de	Süßwaren
leonie.hausmann@ms.de	Molkereiprodukte
leonie.hausmann@ms.de	[NULL]

Fallstudie



DQL 4.5 ■

Zeigen Sie für jeden Mitarbeiter (Emailadresse) alle Bestellungen im System (bestnr, bestellzeitpunkt) mit ihrer jeweiligen Statusbezeichnung (bearbeitungsstatus).

- Es sollen keine Retouren angezeigt werden.
- Sortieren Sie zuerst nach Mitarbeiter email, bearbeitungsstatus aufsteigend, dann nach Bestellzeitpunkt absteigend.

DQL 4.6 ■

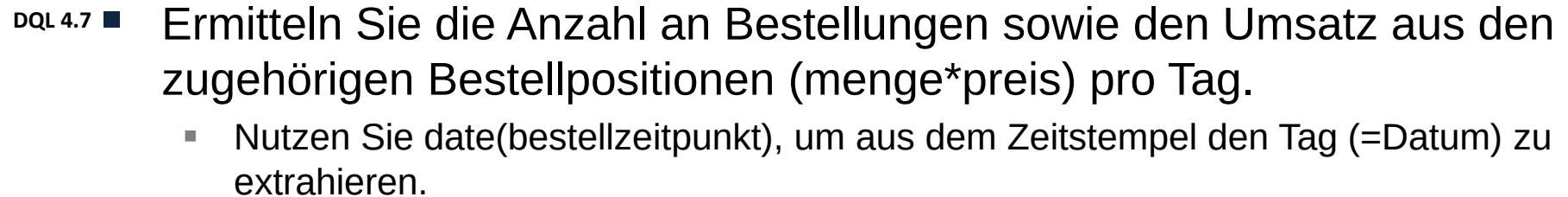
Ermitteln Sie pro Kategorie die Anzahl eindeutiger Produkte und Gesamtmenge der Produkte (Summe von Menge) sowie Gesamtumsatz (menge*preis), die aktuell in Warenkörben der Kunden liegen.

- Liegt kein Produkt in mindestens einem Warenkorb, so soll jeweils 0 bzw. NULL für die Kategorie in den aggregierten Spalten erscheinen.
- Sortieren Sie absteigend nach Gesamtumsatz

```

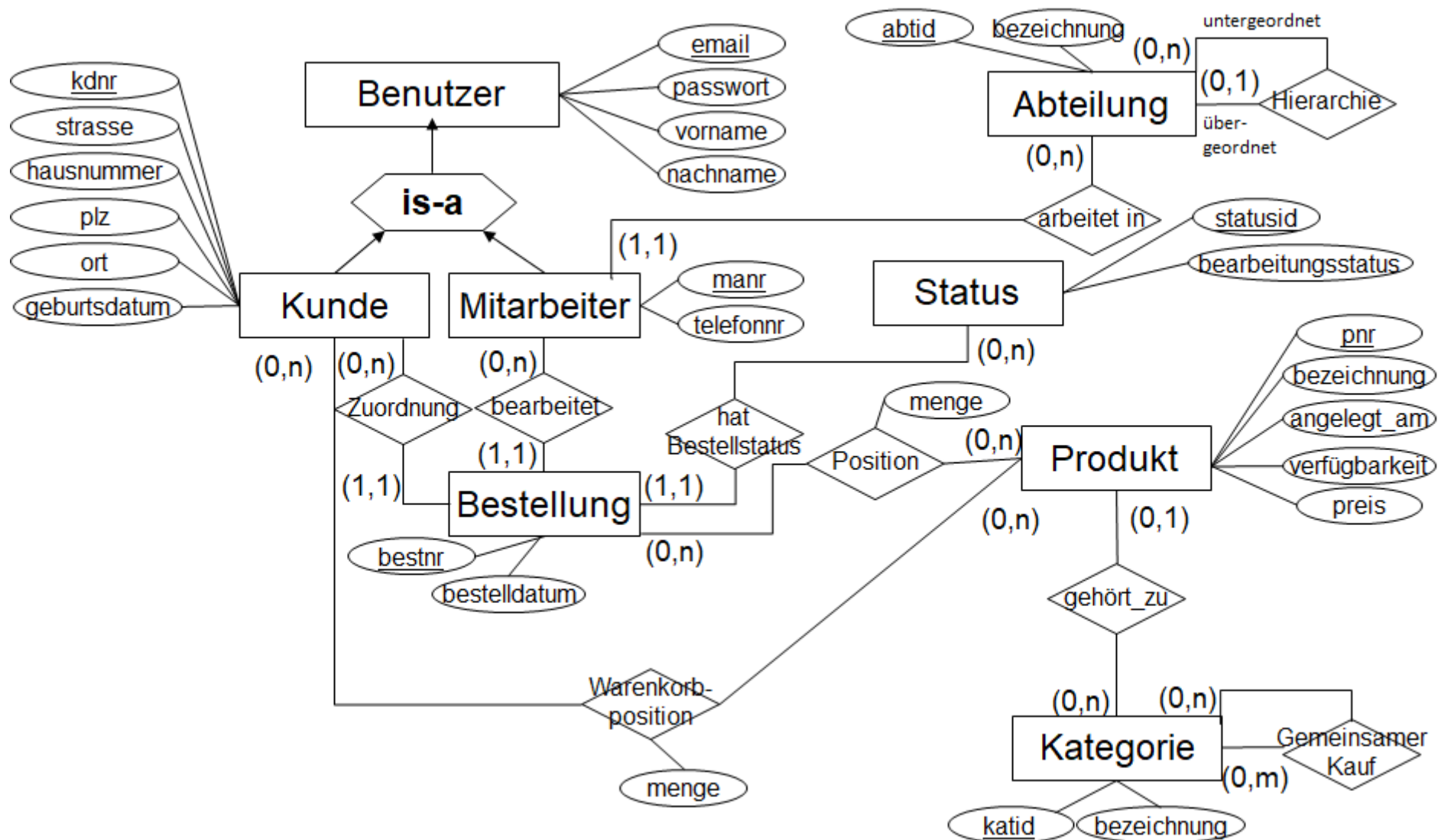
classDiagram
    class Kunde {
        idnr
        adresse
        laurumname
        pnr
        geburtsdatum
    }
    class Mitarbeiter {
        email
        password
        vorname
        nachname
    }
    class Bestellung {
        bestnr
        bestelldatum
    }
    class Produkt {
        menge
        position
        bezeichnung
        eingangspreis
        geburtsdatum
        preis
    }
    class Abteilung {
        abteil
        bezeichnung
        geburtsdatum
    }
    class Status {
        status
        bezeichnung
        geburtsdatum
    }
    class Kategorie {
        kattid
        bezeichnung
    }

    Kunde "0..n" -- "0..n" Mitarbeiter : is-a
    Kunde "0..n" -- "1..1" Bestellung : gurzuehung
    Mitarbeiter "0..n" -- "1..1" Bestellung : bestelldatum
    Bestellung "1..1" -- "0..n" Produkt : menge
    Bestellung "1..1" -- "0..n" Kategorie : position
    Produkt "0..n" -- "0..1" Kategorie : beinhaltet
    Abteilung "0..n" -- "1..1" Bestellung : gurzuehung
    Status "0..n" -- "1..1" Bestellung : gurzuehung
    Kategorie "0..n" -- "0..n" Produkt : beinhaltet
  
```



- Nutzen Sie `year(bestellzeitpunkt)` bzw. `month(bestellzeitpunkt)` um aus dem Zeitstempel die notwendige Werte zu extrahieren.
- Sortieren Sie aufsteigend nach Jahr und Monat.

ERM



Kombination von SELECT-Ergebnissen mit UNION

-
- Das Diagramm zeigt ein ER-Modell für einen Online-Shop. Es enthält die folgenden Entitäten und Attribute:
- Kunde**: kdrnr, strasse, hausnummer, plz, ort, geburtsdatum
 - Mitarbeiter**: email, passwort, vorname, nachname, mnrnr, telefonnr
 - Status**: arbeitert_in, stndtag
 - Bestellung**: bestnr, bestelldatum, zuehung, bearbeitet
 - Produkt**: menge, position, bezeichnung, anzahl, anzahl_in_vorrat, preis
 - Kategorie**: kstnr, bezeichnung
- Die Beziehungen sind wie folgt definiert:
- Kunde** (1) zu **Bestellung** (n) mit der Rolle **Besteller**.
 - Mitarbeiter** (1) zu **Bestellung** (n) mit der Rolle **Bearbeiter**.
 - Bestellung** (1) zu **Produkt** (n) mit der Rolle **Bestellstand**.
 - Status** (1) zu **Produkt** (n) mit der Rolle **menge**.
 - Produkt** (1) zu **Kategorie** (n) mit der Rolle **beziehung**.
 - Kunde** (1) zu **Produkt** (n) mit der Rolle **beziehung**.
 - Mitarbeiter** (1) zu **Produkt** (n) mit der Rolle **beziehung**.
 - Produkt** (1) zu **Kategorie** (n) mit der Rolle **beziehung**.
- Ein rotes 'X' ist über dem Label 'IS-3' zwischen 'Kunde' und 'Mitarbeiter' gesetzt.

```
select email from kunde
union
select email from mitarbeiter
```

ABC email

peter.schulz@ab.de

hans.mueller@xy.de

lea.schmidt@xy.de

leonie.hausmann@ms.de

julia.meier@ab.de

fr.wiedel@ms.de

Agenda

- ☐ Query-Struktur und einfache Abfragen
- ☐ WHERE-Ausdrücke
- ☐ Gruppierung und Aggregation
- ☐ Kombination von Tabellen: JOIN und UNION
- ☒ Subqueries und Common Table Expressions
- ☐ Zusammenfassung und Ausblick

Subqueries: Grundlagen

- Subqueries: SELECT-Statements innerhalb anderer SQL-Statements (SELECT, DELETE, INSERT, ...)
 - Eignen sich sehr gut für dynamische WHERE-Bedingungen und zur Vermeidung komplexer Joins.
- Beispiel
 - Wir möchten alle Produkte sehen, deren Preis über dem Mittelwert liegt.
 - Variante 1 (manuell, nicht empfohlen)
 - Ermittlung des Mittelwerts von „preis“. `select avg(preis) from produkt`
 - Einfügen des Ergebnisses in ein anderes Select-Statement. `select * from produkt where preis > 1.497273`
 - Variante 2 (mit Subquery) `select * from produkt where preis > (select avg(preis) from produkt)`
Subqueries werden in runde Klammern gesetzt
- Subqueries können geschachtelt werden (Subquery in Subquery)

Subqueries: Grundlagen

■ Besonderheiten

- Subqueries werden in runden Klammern spezifiziert
- ORDER BY ist nicht möglich (und auch nicht sinnvoll)
- Liefert eine Subquery **genau einen Wert** zurück, dann werden die üblichen Vergleichsoperatoren (=, <, >, >=, <=, <>, like) verwendet. → **scalar subquery**
 - Beispiel: s.o.

```
select * from produkt
where preis > (select avg(preis) from produkt)
```

- Liefert eine Subquery **mehr als einen Wert für genau eine Spalte** zurück, dann müssen Mengenoperatoren für Vergleiche verwendet werden (bspw. „IN“).
 - Beispiel: Bestellungen an Tagen, an denen auch mindestens ein Produkt angelegt wurde.

```
select *
from bestellung b
where date(bestelldatum) in (select date(angelegt_am) from produkt)
```

Subqueries: Grundlagen

- Verwendung von Spalten der äußeren Query und EXISTS-Statement
 - Innerhalb von Subqueries darf auch auf Spalten der äußeren Query zugegriffen werden. → in dem Fall muss die Subquery pro Wert der Spalte ausgeführt werden
 - Um zu prüfen, ob die Subquery ein Ergebnis zurückliefert oder nicht kann das EXISTS-Statement verwendet werden.
 - Beispiel: selektiere alle Mitarbeiter, die in einer Abteilung arbeiten, über der in der Hierarchie eine andere Abteilung zugeordnet ist:

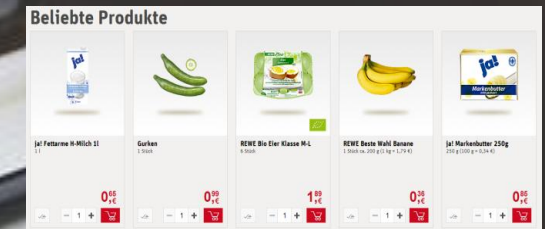
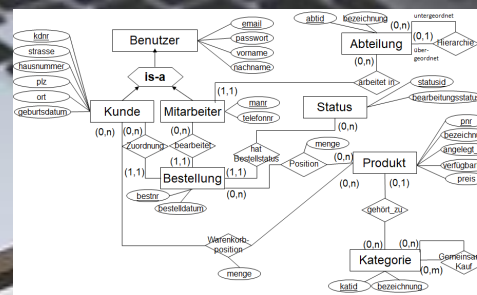
```
select *  
from mitarbeiter m  
where exists (select abtid from abteilung a  
              where a.abtid=m.abtid and  
                    not uebergeordnet_abtid is null)
```

Prüft, ob die Subquery ein
Ergebnis liefert

Verwendung von
Vergleichswerten aus der
äußeren Query
(Zuordnung des richtigen
Mitarbeiters)

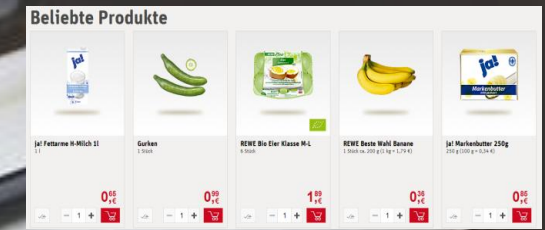
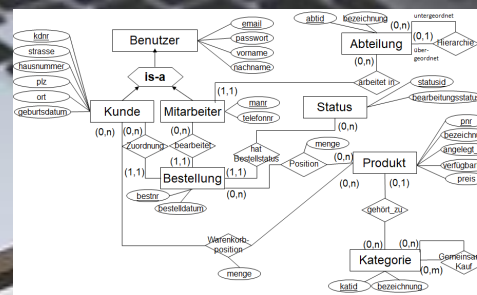
- Achtung: Subqueries können meist durch Joins ersetzt werden!

Fallstudie



- DQL 5.1 ■ Selektieren Sie alle Bestellpositionen, bei denen die Bestellmenge mindestens 95% der maximalen Bestellmenge aller Produkte aufweist.
- DQL 5.2 ■ Selektieren Sie alle Bestellpositionen, bei denen die Bestellmenge mindestens 95% der maximalen Bestellmenge des jeweiligen Produkts aufweist.
- Hinweis: Sie müssen in der Subquery ein WHERE-Statement nutzen und darin die Produktnummer (pnr) aus der „äußeren“ und „inneren“ Query abgleichen.

Fallstudie



DQL 5.3 ■

Wir möchten wissen, ob unsere aktuelle Verfügbarkeit der Produkte ausreichen könnte.

- Zählen Sie dazu je Produkt die Anzahl an Bestellpositionen, bei denen die Bestellmenge größer als die aktuelle Verfügbarkeit war.

DQL 5.4 ■

Wir möchten die hochpreisigen Produkte in unserem Sortiment ermitteln. Selektieren Sie dazu alle Produkte, bei denen der Preis über dem Durchschnittspreis der jeweils zugeordneten Kategorie liegt.

- Hinweis: Sie müssen in der Subquery ein WHERE-Statement nutzen und darin die Kategorien (katid) aus der „äußeren“ und „inneren“ Query abgleichen.
- Ein Join ist nicht nötig.

Subqueries: Insert mit Subquery

- Subqueries können auch in Kombination mit Insert verwendet werden.
 - Beispiel: Übernahme der Daten aus der letzten Bestellung in den Warenkorb

```
-- Definition der Variablen @kunde (Beispielkunden wählen)
```

```
set @kunde = 'leonie.hausmann@ms.de';
```

```
-- Leeren des Warenkorbs des Kunden @kunde
```

```
delete from warenkorbposition where kunde_email = @kunde;
```

```
-- Zwischenschritt: Selektion einer Bestellung von @kunde (mittels Subselect)
```

```
-- @kunde kann als Variable einfach im Select konstant verwendet werden
```

```
select @kunde as kunde, pnr, menge from bestellposition where bestnr =  
(select bestnr from bestellung where kunde = @kunde order by bestnr desc limit 1);
```

```
-- Überführung der Bestellpositionen aus der Bestellung in den aktuellen Warenkorb
```

```
-- Nutzung des Subselects von oben
```

```
insert into warenkorbposition
```

```
select @kunde as kunde, pnr, menge from bestellposition where bestnr =  
(select bestnr from bestellung where kunde = @kunde order by bestnr desc limit 1);
```

```
-- Prüfung der eingefügten Daten
```

```
select * from warenkorbposition
```

```
where kunde_email = @kunde;
```

Common Table Expressions

- Common Table Expressions (CTE) sind temporäre Ergebnisse (einer SELECT-Abfrage), die in einer anderen Abfrage verwendet werden können.
 - Sie stellen somit eine (mächtigere) Alternative zu Subqueries dar.
 - Sie können mit CTEs arbeiten wie mit einer normalen Tabelle. Sie erhalten einen Namen.
 - Die Verknüpfung zwischen CTE und den im SELECT abgefragten Tabellen erfolgt über JOIN.
 - In einem SQL-Statement kann auf **eine oder mehrere temporäre Ergebnismengen** zurückgegriffen werden.
 - Es können also beliebig viele CTEs in ein Statement aufgenommen werden.
 - So können komplexe Abfragen erstellt werden.
 - Die Lesbarkeit ist meist besser als bei Subqueries.

Common Table Expressions

■ CTE – Beispiel

- Gib alle Produkte aus, bei denen der Preis über dem Durchschnittspreis der jeweils zugeordneten Kategorie liegt

Name der temporären Tabelle (CTE)

```
with kategorie_durchschnittspreis_cte AS (  
  select katid, avg(preis) as durchschnittspreis  
  from produkt p  
  group by katid
```

} SELECT zur Generierung der
temporären Tabelle
(die Spaltennamen können
unten verwendet werden)

```
)  
select pnr, bezeichnung, durchschnittspreis  
from produkt  
inner join kategorie_durchschnittspreis_cte using(katid)  
where preis > durchschnittspreis
```

} Das hier ist das eigentliche
SELECT-Statement, bei dem
die „normale“ Tabelle
Produkt mit der CTE
verknüpft wird.

Spalte katid ist sowohl in der
abgefragten Tabelle
„produkt“ als auch in der
CTE vorhanden.

Agenda

- ☐ **Query-Struktur und einfache Abfragen**
- ☐ **WHERE-Ausdrücke**
- ☐ **Gruppierung und Aggregation**
- ☐ **Kombination von Tabellen: JOIN und UNION**
- ☐ **Subqueries und Common Table Expressions**
- ☒ **Zusammenfassung und Ausblick**

Zusammenfassung und Ausblick



- Sie haben nun zahlreiche Methoden zur Abfrage und Analyse von Daten mittels SQL kennengelernt.
- Diese stellen eine gute Grundlage dar, um „Data Science“ in strukturierten Datenquellen und Data Warehouse-Umgebungen zu betreiben.
 - Die Konzepte, die SQL bietet, lassen sich auch schnell auf andere Business Intelligence- und Analytics-Tools übertragen (bspw. Tableau, Power BI oder auch Pivot-Tabellen in Excel).
- In den folgenden Veranstaltungen werden wir uns mit weiteren Funktionen der Konsistenzsicherung in der Datenhaltung befassen.

Vielen Dank für Ihre Aufmerksamkeit!

